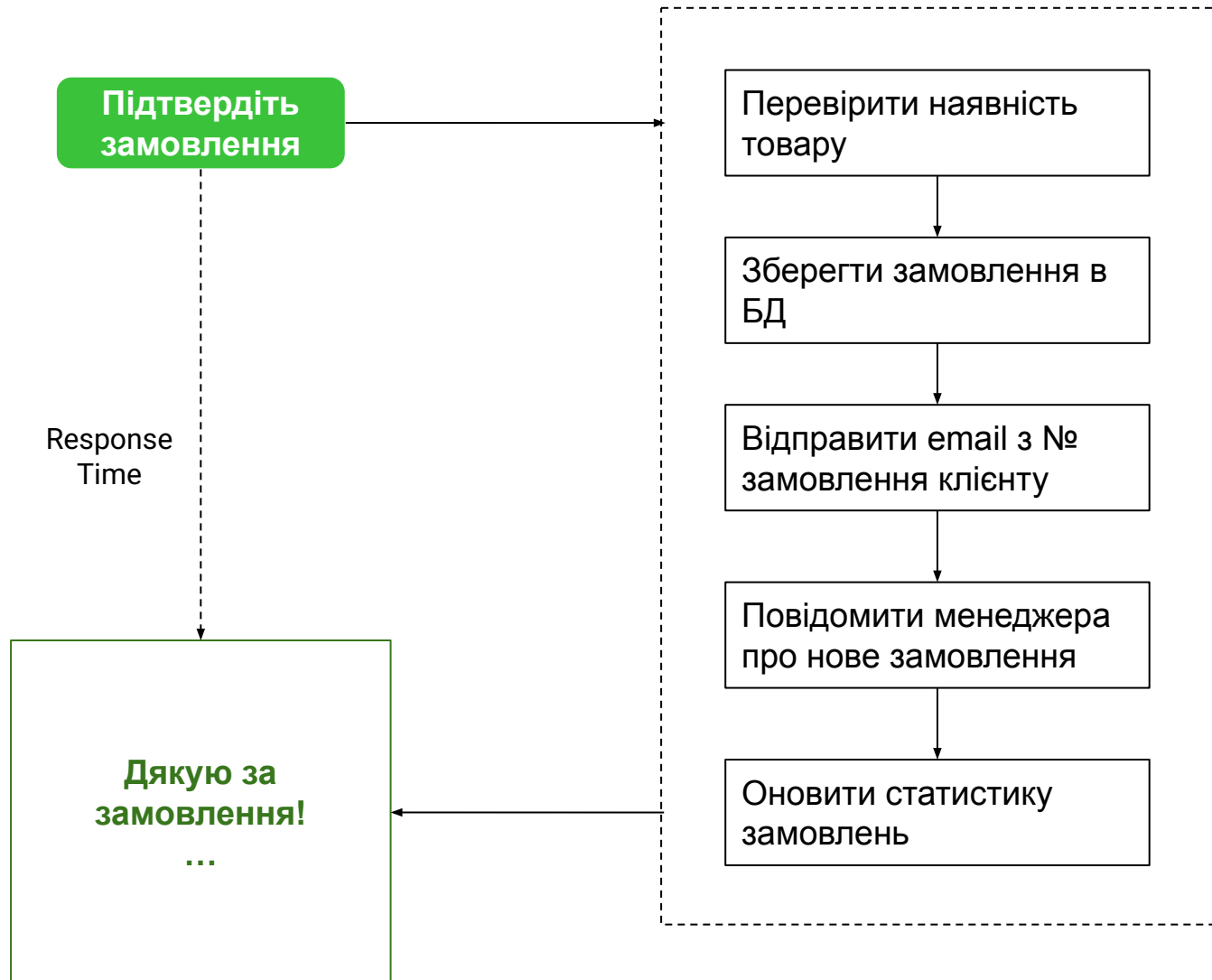


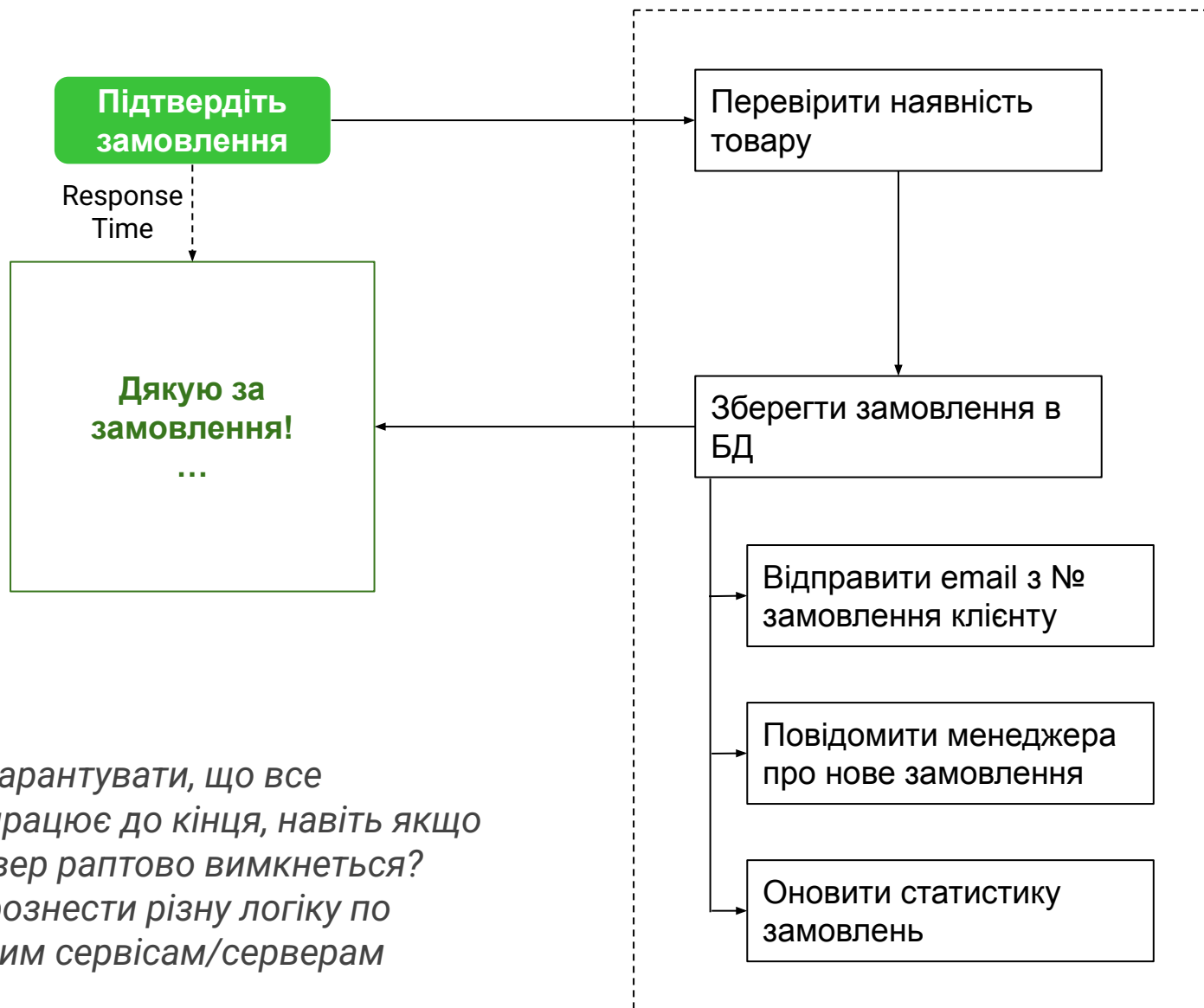
Лекція 18

Message Brokers. Знайомство з RabbitMQ і Apache Kafka

Синхронна обробка запитів



Асинхронна обробка запитів



1. Як гарантувати, що все відпрацює до кінця, навіть якщо сервер раптово вимкнеться?
2. Як рознести різну логіку по різним сервісам/серверам

Message Brokers

Message Broker – програмний застосунок, який відповідає за обмін повідомленнями між компонентами системи.

Обмін повідомленнями відбувається **асинхронно**: один компонент відправляє повідомлення, інший компонент його приймає і обробляє (в якийсь момент часу).

Протоколи і реалізації

Java Message Service (JMS)

- Apache ActiveMQ
- JBoss Messaging
- Oracle WebLogic Server, etc.

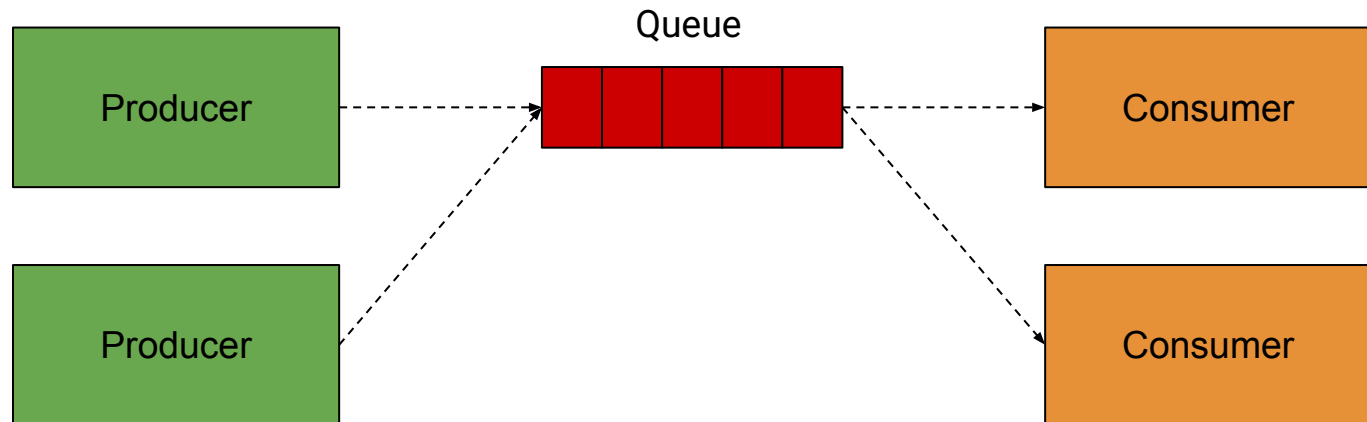
Advanced Message Queuing Protocol (AMQP)

- **RabbitMQ**

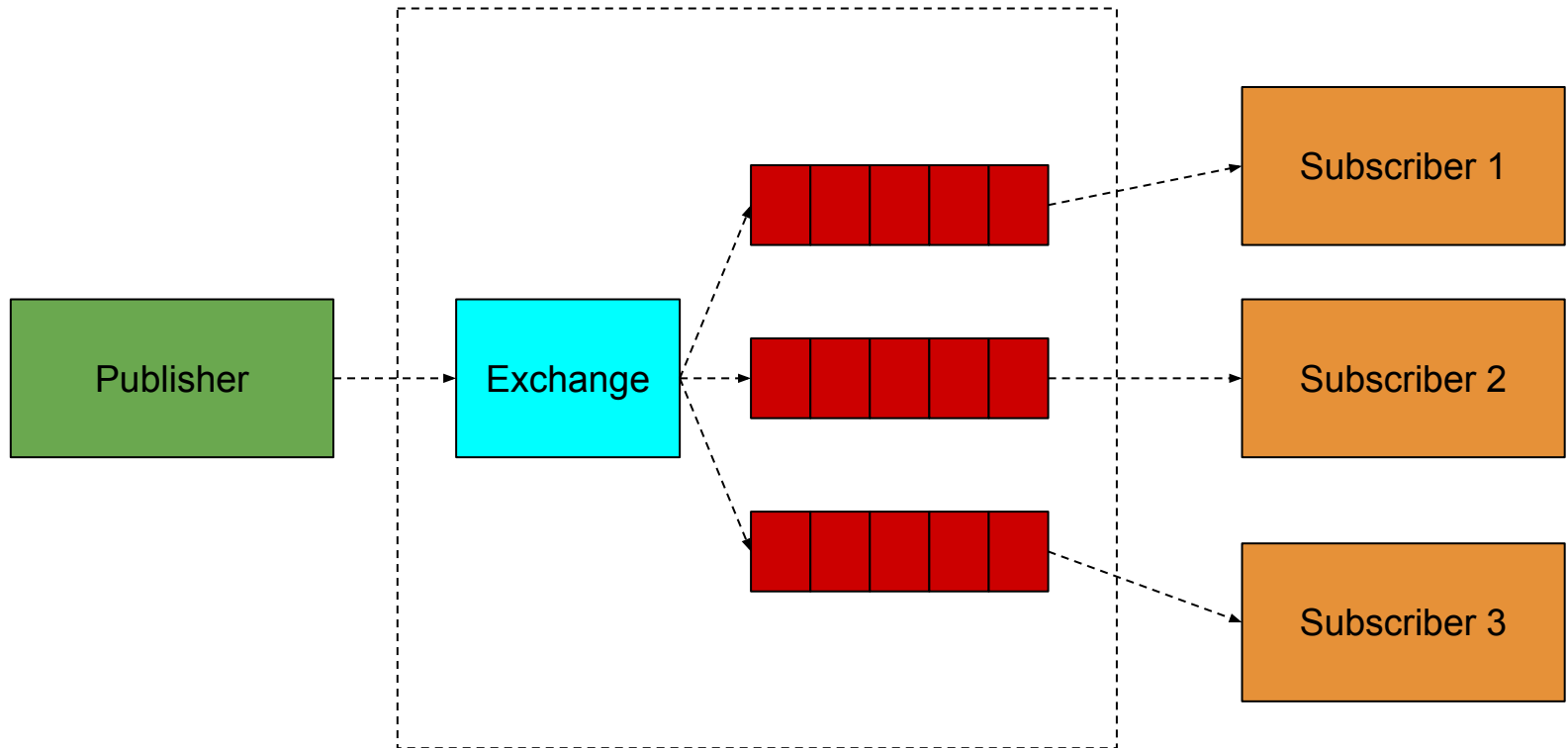
Apache Kafka

Amazon Simple Queue Service (SQS)

Producer/Consumer Model



Publisher/Subscriber Model



Структура повідомлення

- **Metadata** (headers)
- **Payload** (body) – тіло повідомлення у форматі:
 - масив байтів
 - JSON – загальноприйнята практика

Варто уникати великих повідомлень!

Queue Persistence

Повідомлення будуть зберігатися **на диск**, або зберігатись лише в пам'яті.

Non-persistent черги – більш швидкі.

Якщо нам потрібна **гарантована обробка повідомлень**, нам потрібна **persistent (durable)** черга.

Навіть з Queue Persistence не все так просто!



Гарантії доставки повідомлень (від Producer до Message Broker)

Задача: при збої окремого екземпляру брокера або обриві мережевого з'єднання, відправлені producer'ом повідомлення – не повинні губитися.

Гарантії доставки повідомлень:

- **at least once** – якщо не отримано підтвердження успішного отримання, повідомлення буде відправлено повторно і може бути дубльоване
- **at most once** – при відсутності підтвердження отримання, повторної відправки не буде, але повідомлення може бути втрачене
- **exactly once** – при відсутності підтвердження, повідомлення відправляється повторно, але Message Broker задіює додаткові механізми боротьби з дублікатами

Гарантії доставки повідомлень (від Consumer до Message Broker)

Кожне повідомлення має бути гарантовано оброблене, і тільки один раз. Чи можливо це?

Acknowledge – процес підтвердження факту отримання або обробки повідомлення.

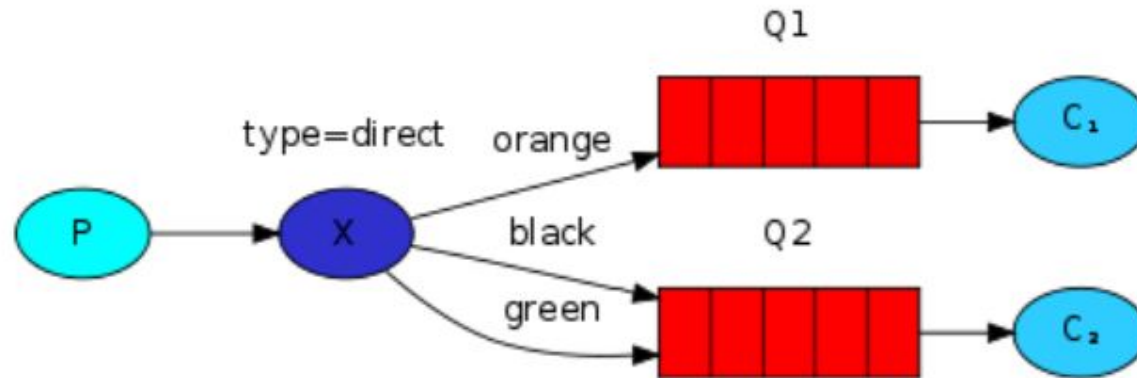
- Його можна робити, як в момент отримання повідомлення, так і в момент завершення обробки
- В RabbitMQ повідомлення після отримання Acknowledge видаляється з черги

В процесі обробки повідомлення, Consumer може раптово впасти або вимкнутися.

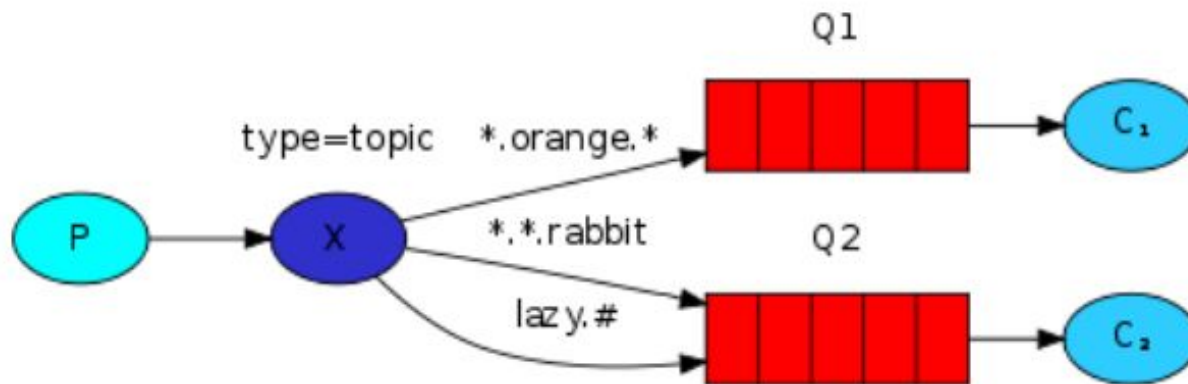
Всередині Consumer може виникнути помилка (наприклад, зовнішній сервіс є недоступним).

В результаті, повідомлення може потрапити в обробку повторно. Тому, потрібно завжди враховувати можливість дублювання!

RabbitMQ – Routing



RabbitMQ – Topic exchange



RabbitMQ – Типи Exchange

direct – направляє повідомлення в чергу за співпадінням routing key

topic – направляє повідомлення у всі черги, для яких співпадає routing key pattern

fanout – направляє повідомлення у всі черги, пов'язані з ним

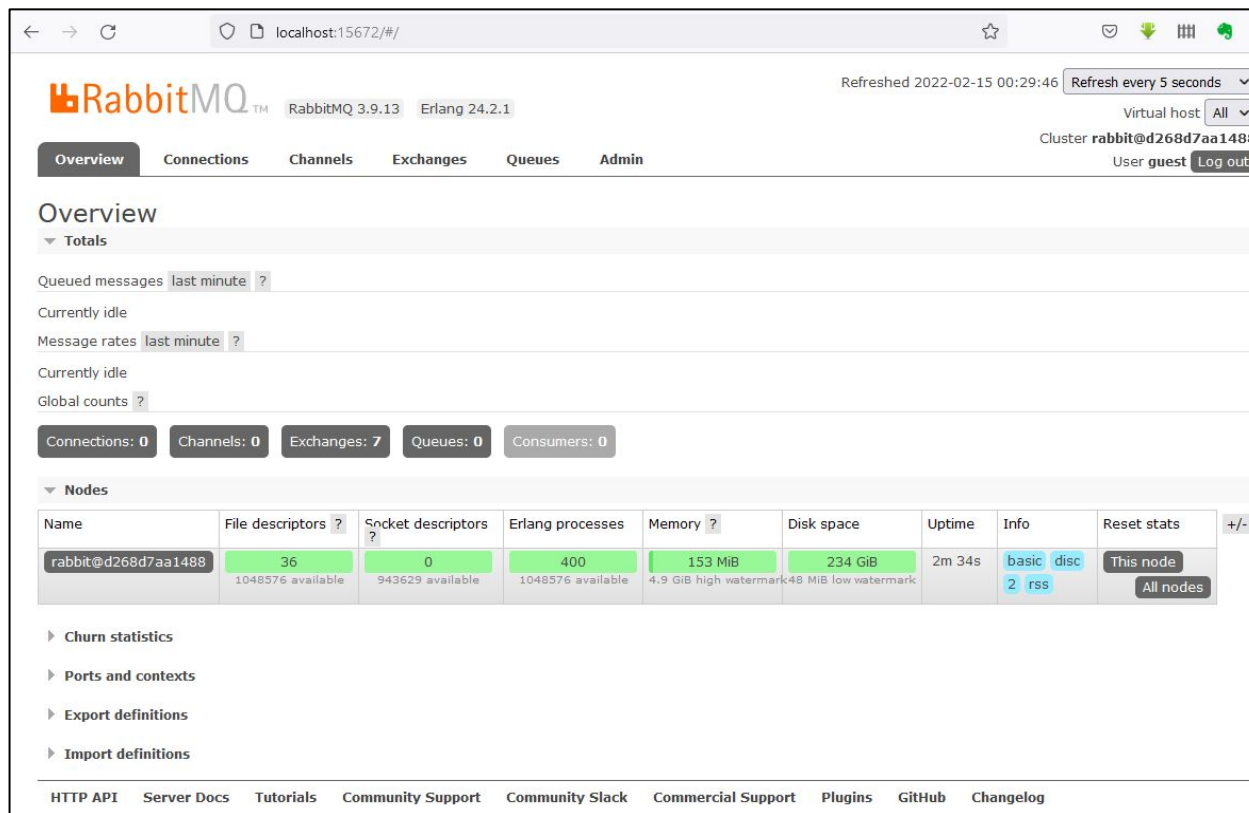
headers – направляє повідомлення в черги у відповідності із заголовками повідомлення

<https://www.rabbitmq.com/tutorials/amqp-concepts.html>

Запускаємо RabbitMQ

```
docker run -it --rm --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:3.9-management
```

RabbitMQ Console



The screenshot displays the RabbitMQ Management Console interface in a web browser. The page title is "RabbitMQ" and the version is "RabbitMQ 3.9.13 Erlang 24.2.1". The interface is refreshed every 5 seconds. The "Overview" tab is selected, showing various statistics and a table of nodes.

Overview

Totals

- Queued messages: last minute ?
- Currently idle
- Message rates: last minute ?
- Currently idle
- Global counts ?

Connections: 0 **Channels: 0** **Exchanges: 7** **Queues: 0** **Consumers: 0**

Nodes

Name	File descriptors ?	Socket descriptors ?	Erlang processes	Memory ?	Disk space	Uptime	Info	Reset stats	+/-
rabbit@d268d7aa1488	36 1048576 available	0 943629 available	400 1048576 available	153 MiB 4.9 GiB high watermark	234 GiB 48 MiB low watermark	2m 34s	basic disc 2 rss	This node All nodes	

Churn statistics

Ports and contexts

Export definitions

Import definitions

[HTTP API](#) [Server Docs](#) [Tutorials](#) [Community Support](#) [Community Slack](#) [Commercial Support](#) [Plugins](#) [GitHub](#) [Changelog](#)

Spring + RabbitMQ

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-amqp</artifactId>
</dependency>
```

RabbitTemplate – для відправки повідомлень

@RabbitListener – для отримання повідомлень

Налаштування з'єднання з RabbitMQ:

У файлі `application.properties` додайте налаштування для підключення до RabbitMQ:

properties

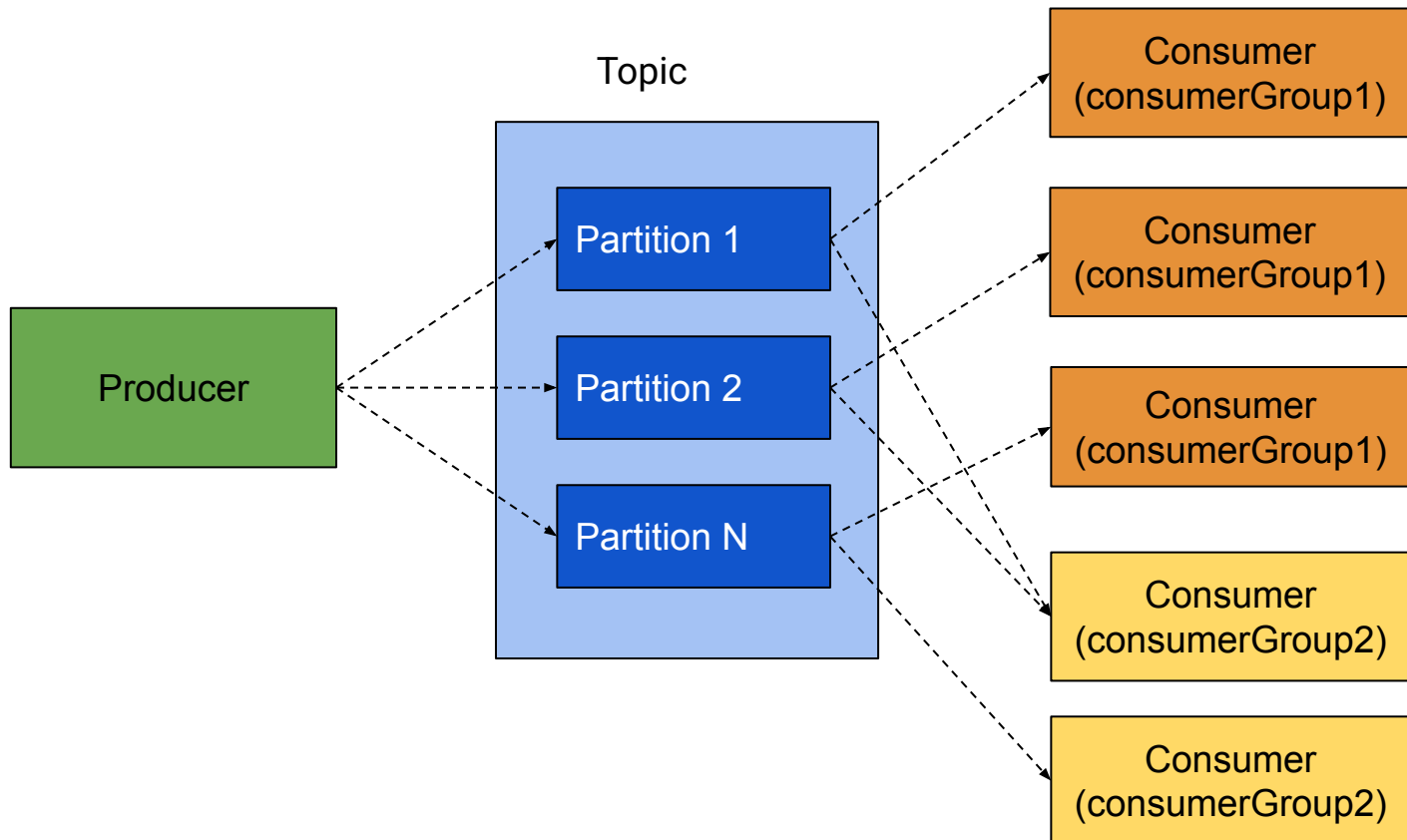
```
spring.rabbitmq.host=localhost
spring.rabbitmq.port=5672
spring.rabbitmq.username=guest
spring.rabbitmq.password=guest
```

```
import org.springframework.amqp.rabbit.annotation.RabbitListener;
import org.springframework.stereotype.Service;

@Service
public class MessageListener {

    @RabbitListener(queues = RabbitMQConfig.QUEUE_NAME)
    public void receiveMessage(String message) {
        System.out.println("Received message: " + message);
    }
}
```

Apache Kafka



Структура повідомлення (Kafka)

- **Key** – **не** унікальний ключ повідомлення. Повідомлення з **однаковим key** будуть потрапляти в **один і той же partition** і, відповідно, оброблятися послідовно
- **Value** – тіло повідомлення; може мати формат масиву байтів, але в більшості випадків – рекомендується використовувати JSON
- **Timestamp**
- **Headers**

Offset – наскрізний номер повідомлення всередині partition

При зчитуванні з **partition** читач робить **offset commit** для тієї **consumer-групи**, до якої він відноситься

- це записується в окремий спеціальний топик
- таким чином consumer-група знає, що вона вже опрацювала, а що ще ні (особливо корисно при рестарті консюмерів)

Кожна consumer-група може обробляти повідомлення незалежно від інших груп

Kafka Retention Period

log.retention.hours – скільки часу повідомлення будуть зберігатися в топіс. 168 годин (7 діб) за замовчуванням.

log.retention.bytes – обмеження на розмір всіх повідомлень в топіс.

Kafka Docker example

```
docker-compose.yml
version: "3"
services:
  zookeeper:
    image: 'bitnami/zookeeper:latest'
    container_name: zookeeper
    ports:
      - '2181:2181'
    environment:
      - ALLOW_ANONYMOUS_LOGIN=yes
  kafka:
    image: 'bitnami/kafka:latest'
    container_name: kafka
    ports:
      - '9092:9092'
    environment:
      - KAFKA_BROKER_ID=1
      - KAFKA_CFG_LISTENERS=PLAINTEXT://:9092
      - KAFKA_CFG_ADVERTISED_LISTENERS=PLAINTEXT://127.0.0.1:9092
      - KAFKA_CFG_ZOOKEEPER_CONNECT=zookeeper:2181
      - ALLOW_PLAINTEXT_LISTENER=yes
    depends_on:
      - zookeeper
```

Відправляємо і отримуємо повідомлення

Створюємо Topic з 2-ма partitions

```
docker exec -ti kafka  
/opt/bitnami/kafka/bin/kafka-topics.sh  
--bootstrap-server localhost:9092 --create  
--replication-factor 1 --partitions 2 --topic  
my-topic
```

Відправляємо повідомлення в topic

```
docker exec -ti kafka  
/opt/bitnami/kafka/bin/kafka-console-producer.sh  
--broker-list localhost:9092 --topic my-topic
```

Читаємо всі вхідні в topic повідомлення

```
docker exec -ti kafka  
/opt/bitnami/kafka/bin/kafka-console-consumer.sh  
--bootstrap-server localhost:9092 --topic my-topic  
--from-beginning
```

Kafka Tool (Offset Explorer)

The screenshot displays the Kafka Tool (Offset Explorer) interface. The left sidebar shows a tree view of the Kafka cluster structure:

- Clusters
 - Local Kafka
 - local
 - Brokers
 - 127.0.0.1:9092
 - Topics
 - paymentReceived
 - _consumer_offset
 - my-topic (selected)
 - Partitions
 - Partition 0
 - Partition 1
 - Consumers

The main panel shows the 'Data' tab for 'my-topic'. It displays a table of messages:

Partition	Offset	Key	Value	Timestamp
0	0		hello from the other side	2023-02-28 10:29:52.740

Below the table, the 'Value' tab is selected, showing the message content: 'hello from the other side'. The status bar at the bottom indicates 'Ready [Messages = 1] [25 Bytes] [133 ms]' and 'Max Messages (per partition) 50'.

Spring + Kafka

pom.xml

```
<dependency>  
    <groupId>org.springframework.kafka</groupId>  
    <artifactId>spring-kafka</artifactId>  
</dependency>
```

<https://www.baeldung.com/spring-kafka>

KafkaOperations: message publishing

```
@Value("${kafka.topic.paymentReceived}")
private String paymentReceivedTopic;

private final KafkaOperations<String, PaymentReceivedMessage> kafkaOperations;

public void someMethod() {
    PaymentReceivedMessage message = ...;
    kafkaOperations.send(paymentReceivedTopic, message.getOrderId(), message);
}
```

application.properties

```
spring.kafka.producer.client-id=myapp
spring.kafka.producer.key-serializer=org.apache.kafka.common.serialization.StringSerializer
spring.kafka.producer.value-serializer=org.springframework.kafka.support.serializer.JsonSerializer
```

@KafkaListener: message consuming

@Component

@RequiredArgsConstructor

```
public class PaymentReceivedListener {  
  
    private final OrderService orderService;  
  
    @KafkaListener(topics = "${kafka.topic.paymentReceived}")  
    public void paymentReceived(PaymentReceivedMessage message) {  
        orderService.processPaymentReceived(message);  
    }  
}
```

application.properties

```
spring.kafka.consumer.group-id=${spring.application.name}  
spring.kafka.consumer.key-deserializer=org.apache.kafka.common.serialization.StringDeserializer  
spring.kafka.consumer.value-deserializer=org.springframework.kafka.support.serializer.JsonDeserializer  
spring.kafka.consumer.properties.spring.json.trusted.packages=dev.profitsoft.jfd.kafkasample.*
```

KafkaAdmin: automatic topics creation

```
@Configuration
public class KafkaTopicConfig {

    @Value(value = "${spring.kafka.bootstrap-servers}")
    private String bootstrapAddress;

    @Value("${kafka.topic.paymentReceived}")
    private String paymentReceivedTopic;

    @Bean
    public KafkaAdmin kafkaAdmin() {
        Map<String, Object> configs = Map.of(
            AdminClientConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapAddress
        );
        return new KafkaAdmin(configs);
    }

    @Bean
    public NewTopic paymentReceivedTopic() {
        return new NewTopic(paymentReceivedTopic, 2, (short) 1);
    }
}
```

Testing Kafka Listeners

```
@SpringBootTest
@DirtiesContext
@ActiveProfiles("test")
@EmbeddedKafka(partitions = 1, brokerProperties = {
    "listeners=PLAINTEXT://localhost:9092", "port=9092" })
public class PaymentReceivedListenerTest {
    @Autowired
    KafkaOperations<String, PaymentReceivedMessage> kafkaOperations;

    ...
}
```

pom.xml

```
<dependency>
    <groupId>org.springframework.kafka</groupId>
    <artifactId>spring-kafka-test</artifactId>
    <scope>test</scope>
</dependency>
```

Test method

@SpyBean

```
private OrderService orderService;
```

@Test

```
void testPaymentReceivedAndPaid() {
```

```
    String orderId = createOrder(1000.0);
```

```
    double payedOffSum = 1000.0;
```

```
    PaymentReceivedMessage message = PaymentReceivedMessage.builder()
```

```
        .orderId(orderId)
```

```
        .sum(payedOffSum)
```

```
        .transactionId("111")
```

```
        .build();
```

```
    kafkaOperations.send(paymentReceivedTopic, orderId, message);
```

```
// As orderService is marked as a @SpyBean, we can use method verify(..) with timeout
```

```
// to wait until method processPaymentReceived is executed
```

```
    verify(orderService, after(5000)).processPaymentReceived(any());
```

```
    OrderDetailsDto orderDetails = orderService.getOrderDetails(orderId);
```

```
    assertThat(orderDetails.getPayedOffSum()).isEqualTo(payedOffSum);
```

```
    assertThat(orderDetails.getStatus()).isEqualTo(OrderStatus.PAYED);
```

```
}
```